CPG Billing & Distribution

Platform architecture, cross-cutting design, API conventions and event contracts

1. Purpose and design principles

CPG Billing & Distribution is an internal, multi-tenant platform for monetizing digital products across existing CPG applications. It owns product/pricing definitions, orders, billing schedules, payment orchestration state, financial records, customer entitlements, digital delivery, license issuance and affiliate attribution.

Payment boundary: CPG owns payment orchestration and its commercial system of record. A payment service provider, acquirer or regulated partner may execute payment rails. Building a payment orchestration layer does not itself make CPG a card network or authorize it to hold/settle third-party funds. Legal and compliance review is a production gate.

1.1 Architectural principles

- Start as a TypeScript modular monolith: one deployment, strict domain modules and explicit service interfaces. Split services only when scale, ownership or isolation justifies it.
- Tenant context is derived from authenticated credentials and membership, never trusted from arbitrary request body fields.
- Financial side effects are idempotent, auditable and reconciled. Do not overwrite financial history to correct errors; post reversing/adjustment entries.
- Use durable asynchronous processing for webhooks, renewals, fulfillment and affiliate commission updates. Avoid in-memory timers for critical work.
- Separate payment, order, invoice, fulfillment and entitlement state machines.
- MongoDB stores domain documents; object storage stores large digital assets; a durable queue executes asynchronous work.
- Use integer currency minor units plus an explicit currency code and currency-precision policy. Do not assume every currency has two decimals.

1.2 Suggested stack

Layer	Technology	Notes							
UI	React + TypeScript	Tenant dashboard, operations console and hosted storefront.	Tenant da shboard, operations console and hosted storefront.	Tenant da shboard, operations console and hosted storefront.	Tenant da shboard, operations console and hosted storefront.	Tenant da shboard, operations console and hosted storefront.	Tenant da shboard, operations console and hosted storefront.	Tenant da shboard, operations console and hosted storefront.	Tenant da shboard, operations console and hosted storefront.
API	Node.js + Express + TypeScript	REST, validation, authorization, orchestration .							
Persistence	MongoDB + Mongoose	Domain collections, indexes, transactions where supported.							
Jobs	BullMQ + Redis or managed queue	Retries, renewals, outbox dispatch, delivery.							

Layer	Technology	Notes							
Files	S3-compatible object storage	Private assets; signed URLs for controlled downloads.							
Contracts	OpenAPI 3.1 + JSON Schema	API source of truth; generate client types where practical.							
Observability	Structured logs, metrics, tracing	Request IDs, event IDs, payment lifecycle correlation.							

2. Cross-cutting HLD

```
Existing CPG applications / React dashboard / hosted storefront
                          |
                @cpg/kubera SDK + REST
                          |
  Authentication -> Tenant Context -> Validation -> Rate Limits
                          |
     +--------------------+--------------------+
     |                    |                    |
  Catalog/Checkout     Billing/Payments     Distribution
     |                    |                    |
     +--------------- Domain Services ---------+
                          |
          MongoDB + Transactional Outbox
                          |
              Durable Queue / Workers
                          |
   Payment Provider | Email | Object Storage | CPG Apps
```

2.1 Domain boundaries

Domain	Owns	Publishes events													
Catalog	Products, immutable price versions, promotions	product.published, price.changed	product.published, price.changed	product.published, price.changed	product.published, price.changed	product.published, price.changed	product.published, price.changed	product.published, price.changed	product.published, price.changed	product.published, price.changed	product.published, price.changed	product.published, price.changed	product.published, price.changed	product.published, price.changed	product.published, price.changed
Checkout/Orders	Checkout sessions, order snapshots, order status	checkout.completed, order.paid													

Domain	Owns	Publishes events													
Payments	Attempts, provider references, refunds, provider events	payment.succeeded, payment.failed, refund.updated													
Billing	Invoices, subscription schedules, usage aggregation	invoice.finalized, subscription.renewed													
Ledger	Balanced journal transactions and account entries	ledger.transaction_posted													
Entitlements	Access grants, expiry and revocation history	entitlement.granted, entitlement.revoked													
Distribution	Asset metadata, delivery attempts, licenses	delivery.completed, license.activated													

Domain	Owns	Publishes events													
Affiliates	Attribution, conversion and commission liability	affiliate.conversion_recorded, commission.adjusted													
Operations	Reconciliation runs, audit logs, support actions	reconciliation.exception_found													

2.2 Common request pipeline

```
HTTP request
  -> requestId / traceId
  -> authentication
  -> resolve tenant and scopes
  -> schema validation (strict; reject unknown fields where appropriate)
  -> idempotency guard for side-effecting operations
  -> domain service + authorization
  -> repository / MongoDB
  -> outbox event in same DB transaction where feasible
  -> response DTO (never return raw Mongoose documents)
```

2.3 Common API conventions

- Base path: /api/v1. JSON uses camelCase, ISO-8601 UTC timestamps, stable public IDs and explicit enum values.
- Use Idempotency-Key for checkout creation, payment initiation, refunds, usage ingestion, credit adjustments and other non-repeatable side effects.
- Return a standard error envelope: { error: { code, message, requestId, details? } }. Never expose stack traces, provider secrets or internal query details.
- Use cursor pagination for large lists: ?limit=50&after;=. Enforce maximum page size.
- Use ETag/version or expectedVersion for concurrent administrative updates where stale writes matter.
- Use webhooks for asynchronous updates; signature verification and deduplication are mandatory.

2.4 Common security controls

- Tenant-scoped unique indexes and repository methods that require tenantId. Avoid repository methods that fetch by ID alone.
- Role/scopes for platform operator, tenant admin, developer, finance operator and read-only analyst.
- Secrets in a managed secrets store; rotate provider credentials and API keys.
- Do not store raw PAN/CVV. Use provider-hosted checkout/tokenization unless a separately approved PCI architecture exists.
- Encrypt sensitive data in transit and at rest; redact email, tokens, license keys and payment references from logs.
- Rate-limit public checkout, license activation, auth-sensitive routes and usage ingestion.
- Audit price edits, refunds, credit adjustments, entitlement overrides, provider config and affiliate commission changes.

3. Shared data and DTO conventions

3.1 Shared TypeScript primitives

```
export type PublicId =
  | `ten_${string}` | `prod_${string}` | `price_${string}`
  | `cus_${string}` | `ord_${string}` | `pay_${string}`
  | `sub_${string}` | `inv_${string}` | `ent_${string}`;

export type CurrencyCode = string; // Validate against configured ISO-4217 set.
export type ISODateTime = string;  // Runtime validate as RFC3339/ISO-8601.
export type TenantContext = {
  tenantId: string;
  principalId: string;
  scopes: readonly string[];
  requestId: string;
};

export type Page<T> = {
  data: T[];
  page: { nextCursor: string | null; hasMore: boolean };
};

export type ApiErrorBody = {
  error: {
    code: string;
    message: string;
    requestId: string;
    details?: Array<{ path: string; code: string; message: string }>;
  };
};
```

3.2 Validation rules

- Validate at runtime on the server using a schema library such as Zod, TypeBox/Ajv or an equivalent approved standard; TypeScript types alone do not validate untrusted JSON.
- Use strict DTO allow-lists; never pass client payloads directly to Mongoose update operators.
- Bound string lengths, array sizes, quantities, metadata size, file sizes and pagination limits.
- Validate currency and amount precision; reject negative quantities unless a documented adjustment endpoint explicitly supports them.
- Treat metadata as untrusted and size-limited; do not allow secrets or reserved system keys.
- Generate API types from OpenAPI or shared schema packages to reduce drift; publish a compatibility policy.

4. Cross-domain sequence: paid order to delivery

```
sequenceDiagram
  participant UI as CPG UI
  participant SDK as @cpg/kubera
  participant API as Express API
  participant DB as MongoDB
  participant PSP as Payment Provider
  participant Q as Durable Worker
  participant ENT as Entitlements
  participant DIST as Distribution

  UI->>SDK: checkout.createSession(dto)
  SDK->>API: POST /checkout/sessions
  API->>DB: Save session + immutable order snapshot
  API->>PSP: Create provider payment/session
  PSP-->>API: Provider reference / checkout URL
  API-->>SDK: CheckoutSessionDTO
  SDK-->>UI: Typed session
  PSP->>API: Signed webhook
  API->>API: Verify signature + deduplicate
  API->>DB: Persist provider event + state/outbox
  API-->>PSP: 2xx after durable acceptance
  Q->>DB: Claim outbox event
  Q->>ENT: Grant entitlement idempotently
  Q->>DIST: Fulfill product idempotently
  DIST->>DB: Save delivery result
  Q->>DB: Mark event processed
```

5. Reliability, operations and release gates

- Provider timeout is an uncertain outcome, not automatically a failed payment. Query/reconcile before retrying with a new provider operation.
- Outbox workers use leases/visibility timeouts, bounded exponential backoff with jitter, and a dead-letter state. Operators can replay after inspection.
- Daily minimum reconciliation compares provider transactions/settlements with CPG payment and ledger records; uncertain transactions should reconcile more frequently.
- Monitor payment success/failure rates, webhook lag, renewal lag, outbox age, dead-letter count, delivery failure rate, credit conflicts and reconciliation exceptions.
- Set an SLO after load testing. Suggested initial targets: 99.9% internal API availability and p95 under 300 ms for normal reads, excluding external provider latency.
- Before launch: threat model, tenant-isolation tests, webhook replay tests, concurrency tests, disaster-recovery exercise and finance sign-off.

6. Suggested repository layout

```
apps/
  api/src/modules/{catalog,checkout,orders,payments,billing,ledger,
                   entitlements,distribution,licenses,affiliates,operations}/
  dashboard/src/{app,routes,features,components}/
  storefront/src/{routes,components,checkout}/
packages/
  kubera/src/{client,errors,resources,types}/
  contracts/src/{schemas,dto,events}/
  config/{eslint,typescript}/
  testkit/{fixtures,mocks}/
```